Autonomous Visual Tracking and Control System

Suvrojit Ghosh

Abstract

This documentation describes a MATLAB-based visual servoing algorithm designed for autonomous navigation and target centering. The system processes 160 x 120 RGB video frames to detect "pure red" objects, calculates their horizontal centroid, and generates proportional steering commands. By comparing the target's position to the image center, the controller produces a correction signal to align the vehicle's heading. In the context of Aerial Robotics Kharagpur (ARK), this fundamental logic is critical for tasks such as red-line following, automated landing on colored pads, or tracking specific colored payloads during flight operations.

I. INTRODUCTION

The primary objective of this project is to implement a "Search and Track" capability for an autonomous system. The approach utilizes a color-thresholding method to isolate a target and a **Proportional (P)** controller to adjust the vehicle's orientation. We filter pixels based on RGB intensity to ensure the red channel is dominant, effectively rejecting environmental noise from green and blue channels (such as grass or sky).

II. PROBLEM STATEMENT

Given an input RGB image of size 160 x 120, the system must perform the following:

1. **Pixel Classification:** Identify pixels where R > 150, G < 100, and B < 100.
2. **Spatial Localization:** Calculate the horizontal centroid (cx) of all detected pixels.
3. **Error Calculation:** Determine the deviation relative to the image center (W/2 = 80).
4. **Command Generation:** Output a constant forward velocity (X = 1.25) and a steering correction (Y) to minimize the tracking error.

III. RELATED WORK

Color-based tracking is a staple in robotics for its low computational overhead. While more advanced methods like HSV (Hue, Saturation, Value) thresholding offer better robustness to lighting, the RGB thresholding used here is highly efficient for low-latency control loops on embedded microcontrollers. This logic follows the standard principles of feedback control loops used in industrial line-following robots.

IV. INITIAL ATTEMPTS

Initial iterations of the script lacked the safety check for the "count" variable. Without the "if count < 5" condition, the system would attempt to divide by zero when no red object was in view, leading to immediate controller failure. The current implementation defaults to X = 0, Y = 0 (safe stop) if an insufficient number of red pixels are detected, ensuring system stability.

V. FINAL APPROACH

The final implementation utilizes a nested loop for image scanning and a linear control law for navigation.

1. Detection Logic

The algorithm iterates through every pixel coordinate (y, x) and applies a strict red-dominance filter:

(R > 150) AND (G < 100) AND (B < 100)

2. Centroid Calculation

The horizontal centroid (cx) is derived from the sum of the x-coordinates of all valid pixels:

cx = (Sum of all x) / (Total count)

3. Control Law

The tracking error is defined as the distance from the dead-center of the frame:

error = cx - 80

The steering command (Y) is generated using a proportional gain (Kp):

Y = -Kp * error

Where Kp = 0.0001. The negative sign ensures the steering is "corrective"—turning left if the object is to the right of the center.

VI. RESULTS AND OBSERVATION

Parameter	Value	Description
Forward Velocity (X)	1.25	Constant speed maintained during tracking.
Steering Gain (Kp)	0.0001	Conservative correction factor.
Min. Detection Area	5 pixels	Threshold to prevent noise-triggering.

Observation: The gain of 0.0001 is quite conservative. In high-speed ARK flight tests, this might result in "lazy" centering. However, it prevents the aggressive oscillations often seen in high-gain proportional systems.

VII. FUTURE WORK

Future improvements include transitioning from RGB to **HSV Color Space** to improve detection under varying sunlight conditions. Additionally, implementing a **PID (Proportional-Integral-Derivative)** controller would allow for faster centering without overshooting the target.

CONCLUSION

This algorithm provides a reliable baseline for visual navigation at ARK. By isolating specific color signatures and applying a proportional response, the vehicle can successfully track targets. The inclusion of safety thresholds makes the code robust for real-world deployment where targets may occasionally leave the field of view.